

Generative Systems

ECMM422

Fabrizio Costa

Winter 2026



Outline

- 1 Generative Systems
- 2 Autoencoders
- 3 Generative Adversarial Networks (GANs)
- 4 Variational AutoEncoders
- 5 Transformers
- 6 Diffusion models

- ▶ In Machine Learning we can distinguish two approaches, the generative approach and the discriminative approach.
- ① a generative model is a model of the **joint** probability distribution $P(X, Y)$ on the observable X and target Y
- ② a discriminative model is a model of the **conditional** probability $P(Y | X = x)$ of the target Y given an observation x

- ▶ Sometimes one can find the following alternative (symmetrical) definition:
 - ① a generative model is a model of the conditional probability of the **observable** X , given a target y : $P(X \mid Y = y)$
 - ② a discriminative model is a model of the conditional probability of the **target** Y , given an observation x : $P(Y \mid X = x)$

- ▶ A generative algorithm models **how** the data was generated in order to categorise a signal
- ▶ ...it asks the question: based on an hypothesised generation mechanism, which category is most likely to generate this signal?
- ▶ A discriminative algorithm does not care about how the data was generated
- ▶ ...it simply wants to **categorise** a given signal, i.e. it learns $p(y|x)$ directly from the data so to correctly classify data

- ▶ Informally:
 - ▶ Generative models can generate new data instances
 - ▶ Discriminative models discriminate between different (previously unseen) data instances
- ▶ A generative model could generate new photos of animals that look like real animals, while a discriminative model could tell a dog from a cat
- ▶ A generative model includes the distribution of the data itself, and tells you **how likely** a given example is.
For example, models that predict the next word in a sequence are typically generative models because they can assign a probability to any given sentence
- ▶ A discriminative model ignores the question of whether a given instance is likely, and just tells you how likely a label is for a given instance

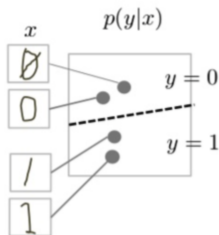
Generative Models Are Difficult

- ▶ Generative models try to solve a more difficult problem than the problem addressed by discriminative models.
Generative models have to model "more".
 - ▶ A generative model for images might capture correlations like "things that look like boats are probably going to appear near things that look like water" and "eyes are unlikely to appear on foreheads."
 - ▶ A discriminative model might learn the difference between "sailboat" or "not sailboat" by just looking for a few tell-tale patterns (e.g. mainly blue color on the lower part of the image) It could ignore many of the correlations that the generative model must get right.

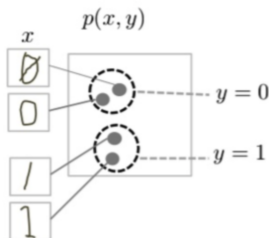
Generative Models Are Difficult

- ▶ Discriminative models try to draw boundaries in the data space, while generative models try to model how data is placed throughout the space.

- Discriminative Model



- Generative Model



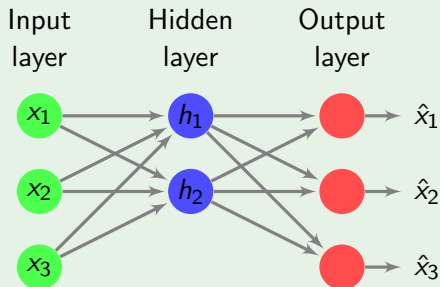
Types of generative models

- ▶ Gaussian mixture model (and other types of mixture model)
- ▶ Hidden Markov model
- ▶ Probabilistic context-free grammar
- ▶ Bayesian network (e.g. Naive Bayes, Autoregressive model)
- ▶ Boltzmann machine (e.g. Restricted Boltzmann machine, Deep belief network)
- ▶ Variational autoencoder
- ▶ Generative adversarial network
- ▶ Flow-based generative model
- ▶ Energy based model
- ▶ Diffusion model

Outline

- 1 Generative Systems
- 2 Autoencoders
- 3 Generative Adversarial Networks (GANs)
- 4 Variational AutoEncoders
- 5 Transformers
- 6 Diffusion models

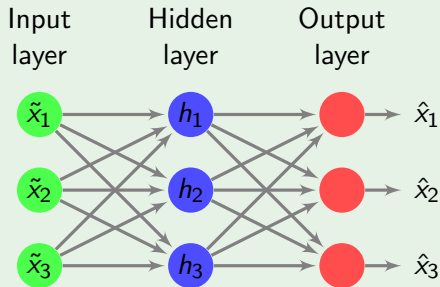
Autoencoders



- ▶ The error function is the difference between the output and input vectors:
$$L(x_i) = \frac{1}{2}(x_i - \hat{x}_i)^2$$
- ▶ learning can be done using backpropagation and stochastic gradient descent.

- ▶ An autoencoder attempts to learn a pair of mappings **encoding** and **decoding** the input signal.
- ▶ The hidden layer contains an encoded representation that preserves information in x .
- ▶ If $|H| < |X|$, then the representation in H is compressed compared to X (otherwise the trivial mapping is a global optimum).

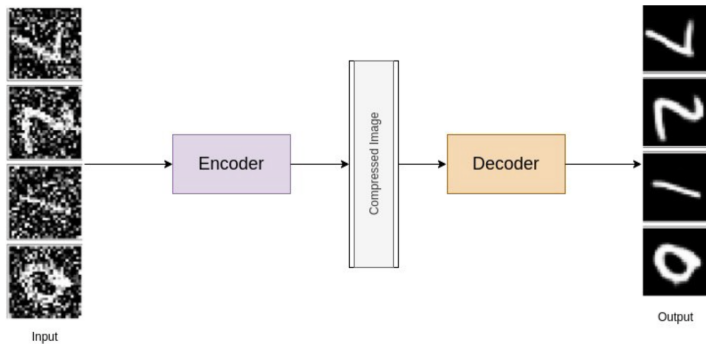
Denoising autoencoders



- ▶ The error function is the difference between the output and input vectors:
$$L(x_i) = \frac{1}{2}(x_i - \hat{x}_i)^2$$
- ▶ training vectors are corrupted with noise before being fed to the network: $\tilde{x} = \chi_q(x)$

- ▶ The denoising autoencoder optimises a dual objective:
 - ▶ encoding the input while preserving information
 - ▶ remove corruption from the input signal
- ▶ The corruption function $\chi_q(x)$ sets q randomly chosen variables to zero.

Denoising Autoencoders on MNIST



- ▶ Classical autoencoders attempt to learn good feature by compressing the signal dimension, but evidence from neuroscience indicates that sparse, overcomplete encodings may be beneficial.
- ▶ A sparse autoencoder tries to optimise the dual objective:
 - ▶ encoding the input while preserving information
 - ▶ find an encoding where most of the hidden units are zero for any given input vector (sparsity).

Sparse autoencoders

Many approach in practice:

- 1 L_1 regularization: penalize the size of the vector of activations a in layer h for observation i

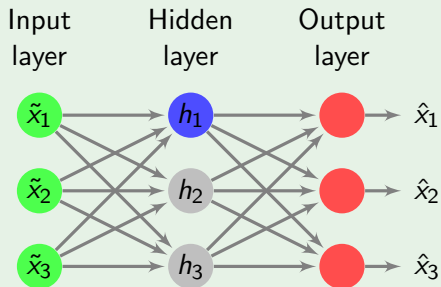
$$L(x, \hat{x}) + \lambda \sum_i |a_i^{(h)}|$$

- 2 add a regularisation term to the objective function (which may make the backpropagation more complex)
define a sparsity parameter ρ which denotes the average activation of a neuron over a collection of samples, consider $\hat{\rho}_j$ as the expected sparsity for neuron j in layer h over a batch of examples

$$L = L(x, \hat{x}) + \sum_{j=1}^{|H|} KL(\rho || \hat{\rho}_j)$$

- 3 artificially zero all hidden units aside from the k strongest (simplest approach).

k-sparse autoencoders

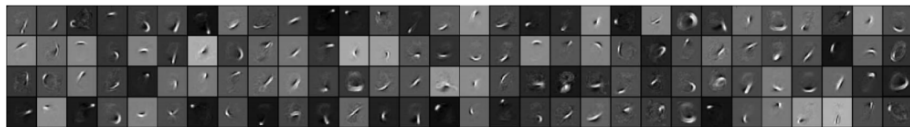


- ▶ The error function is the difference between the output and input vectors:
$$L(x_i) = \frac{1}{2}(x_i - \hat{x}_i)^2$$
- ▶ the $n^{(h)} - k$ weakest hidden units are zeroed (in the picture $k = 1$).

- ▶ The denoising autoencoder optimises a dual objective:
 - ▶ encoding the input while preserving information
 - ▶ remove corruption from the input signal
- ▶ The corruption function $\chi_q(x)$ sets q randomly chosen variables to zero.

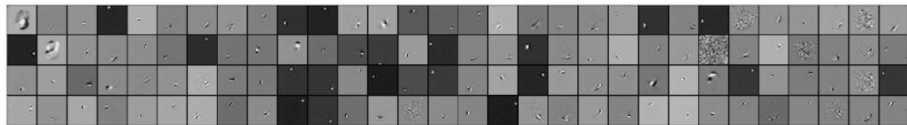
k -sparse autoencoders on MNIST

- ▶ k -sparse autoencoder trained on the MNIST (hand written digits) dataset
- ▶ 1,000 hidden units.
- ▶ only k units at a time can be non-zero.

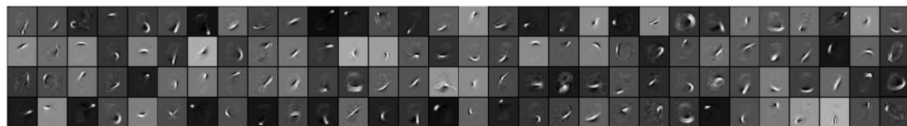


(b) $k = 40$

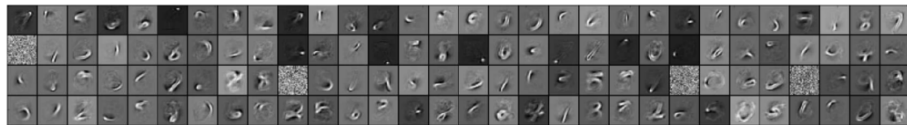
Alireza Makhzani, Brendan Frey (2014). “ k -Sparse Autoencoder” *International Conference on Learning Representations (ICLR)*.



(a) $k = 70$



(b) $k = 40$



(c) $k = 25$

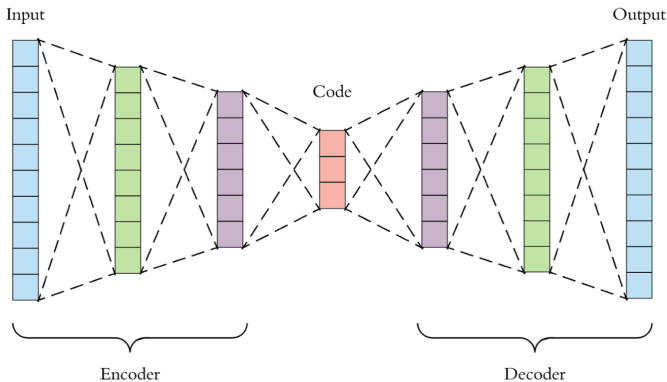


(d) $k = 10$

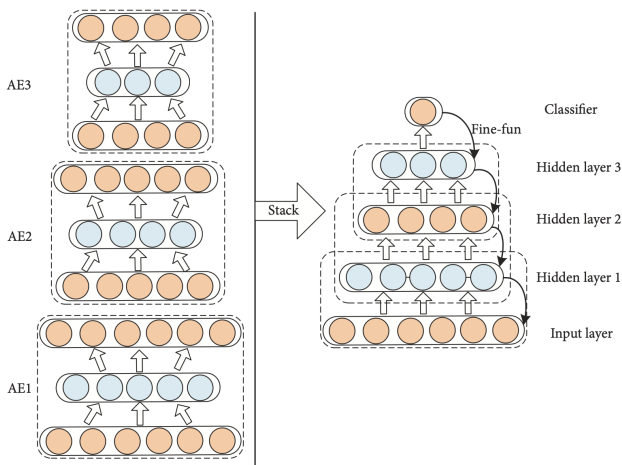
Stacking autoencoders

Question

Until now we have seen autoencoders as a special case of neural networks, with a limited number of layers; how can we build a deep hierarchy from those?



- It is possible to build **stacked autoencoders**, where each layer $l + 1$ is learnt by autoencoding the representation (hidden layer) from the previous autoencoding layer l .



- ▶ The learnt deep representation can then be used for classification or regression by adding a standard fully connected layer at the last layer.
- ▶ The whole, deep network can then be **fine-tuned** for the task using standard back-propagation. This use is called **unsupervised pre-training**.
- ▶ Unsupervised pre-training is efficient because the AA are trained efficiently **layer-wise** allowing to initialise the NN weights to good features (better than the typical random weights).

Outline

- 1 Generative Systems
- 2 Autoencoders
- 3 Generative Adversarial Networks (GANs)
- 4 Variational AutoEncoders
- 5 Transformers
- 6 Diffusion models

Generative Adversarial Networks (GAN)

Remember how adversarial examples could be generated to fool a classifier?

Idea

Model the data as a competitive process between a *Discriminator* (D), which tries to recognise genuine training examples, and a *Generator* (G), which tries to generate fake examples to fool the first.

- ▶ The Discriminator can be a classical neural network with a two-class output $D : \mathbb{R}^K \rightarrow \{0, 1\}$.
- ▶ the Generator takes a random seed z as input and outputs a vector of the same size as the generator's input x .

Generative Adversarial Networks (GAN)

Remember how adversarial examples could be generated to fool a classifier?

Idea

Model the data as a competitive process between a *Discriminator* (D), which tries to recognise genuine training examples, and a *Generator* (G), which tries to generate fake examples to fool the first.

Remember the cross-entropy :

$$L(D) = \sum_{n=1}^N t_n \log D(x_n) + (1 - t_n) \log(1 - D(x_n))$$

Generative Adversarial Networks (GAN)

Remember how adversarial examples could be generated to fool a classifier?

Idea

Model the data as a competitive process between a *Discriminator* (D), which tries to recognise genuine training examples, and a *Generator* (G), which tries to generate fake examples to fool the first.

Remember the cross-entropy :

$$L(D) = \sum_{n=1}^N t_n \log D(x_n) + (1 - t_n) \log(1 - D(x_n))$$

If we consider that $t_n = 1$ for all examples in the training set, and that $t_n = 0$ for all $G(z) \forall z$, we effectively optimise the following min-max problem:

$$\min_G \max_D L(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_{data}(z)} [\log(1 - D(G(z)))]$$

Minimax GAN Loss

- ▶ Minimax GAN loss refers to the minimax simultaneous optimization of the discriminator and generator models.
- ▶ The min and max refer to the minimization of the generator loss and the maximization of the discriminator's loss

$$\min_G \max_D L(D, G)$$

- ▶ the discriminator seeks to maximize the average of the log probability of real images and the log of the inverse probability for fake images; the discriminator maximizes

$$\log D(x) + \log(1 - D(G(z)))$$

- ▶ The generator seeks to minimize the log of the inverse probability predicted by the discriminator for fake images. This has the effect of encouraging the generator to generate samples that have a low probability of being fake; the generator minimizes

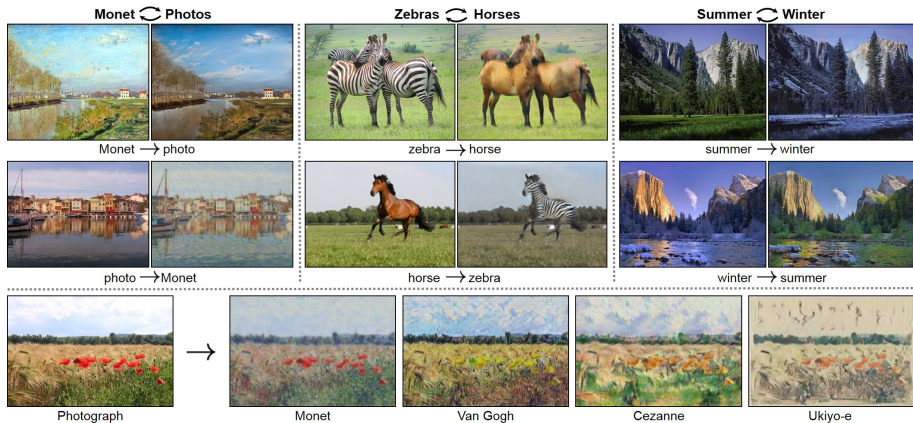
$$\log(1 - D(G(z)))$$

Examples

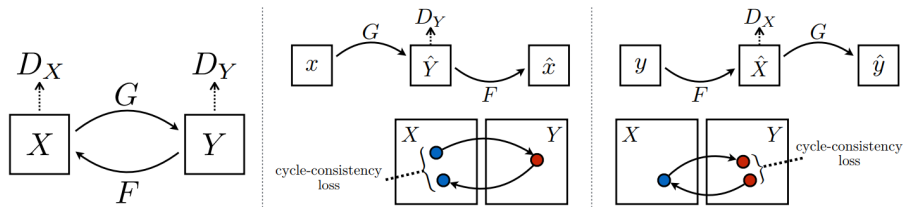


Ian Goodfellow Jean Pouget-Abadie Mehdi Mirza Bing Xu David Warde-Farley
Sherjil Ozair Aaron Courville Yoshua Bengio. "Generative Adversarial Nets"
Advances in Neural Information Processing Systems 27 (NIPS 2014)

CycleGAN: Style Transfer



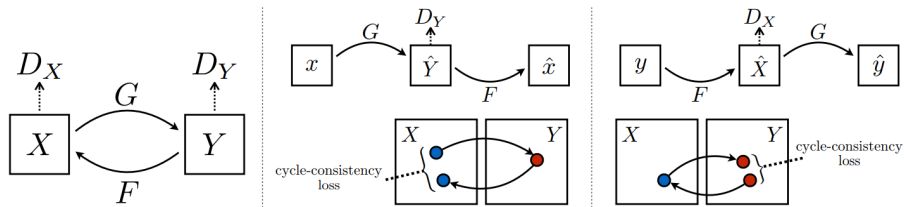
CycleGAN: Style Transfer



Loss

- ▶ Learn mapping function between two domains X and Y given training samples $\{x_i\}_{i=1}^N$ and $\{y_j\}_{j=1}^M$ where $x_i \in X$ and $y_j \in Y$.
- ▶ Two mapping functions: $G : X \rightarrow Y$ and $F : Y \rightarrow X$
- ▶ Two discriminators D_X and D_Y

CycleGAN: Style Transfer

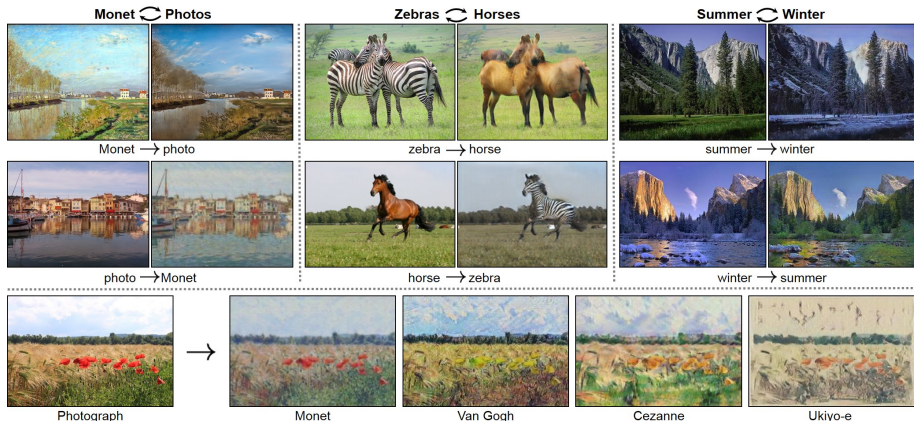


Loss

The overall cost optimised by the approach is:

$$\mathcal{L}(G, F, D_X, D_Y) = \mathcal{L}_{GAN}(G, D_Y, X, Y) + \mathcal{L}_{GAN}(F, D_X, Y, X) + \lambda \mathcal{L}_{cyc}(G, F)$$

Application: Style transfer and CycleGAN



Jun-Yan Zhu*, Taesung Park*, Phillip Isola, Alexei A. Efros Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks *In ICCV 2017*.

StyleGAN: <https://github.com/NVlabs/stylegan>

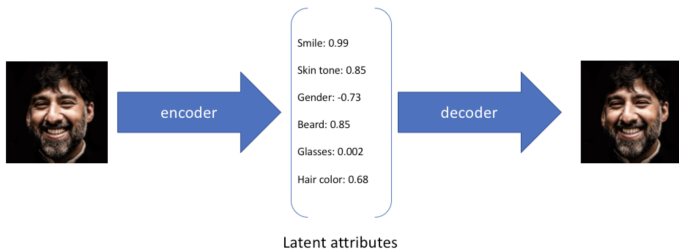


Outline

- 1 Generative Systems
- 2 Autoencoders
- 3 Generative Adversarial Networks (GANs)
- 4 Variational AutoEncoders**
- 5 Transformers
- 6 Diffusion models

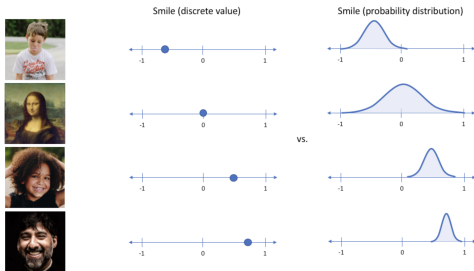
Variational AutoEncoders (VAE)

- ▶ A variational autoencoder (VAE) provides a probabilistic manner for describing an observation in latent space
- ▶ Rather than building an encoder which outputs a single value to describe each latent state attribute, we'll formulate our encoder to describe a probability distribution for each latent attribute.



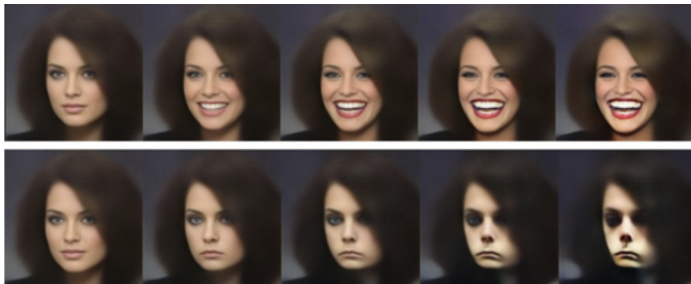
Variational AutoEncoders (VAE)

- ▶ We can represent each latent attribute for a given input as a probability distribution
- ▶ When decoding from the latent state, we'll randomly sample from each latent state distribution to generate a vector as input for our decoder model probability distribution for each latent attribute.



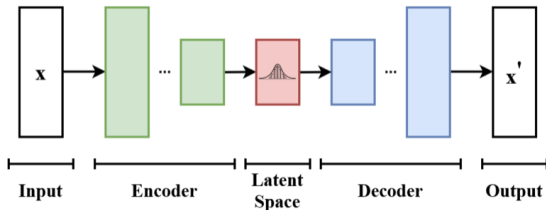
Variational AutoEncoders (VAE)

- By constructing our encoder model to output a range of possible values (a statistical distribution) from which we'll randomly sample to feed into our decoder model, we're essentially enforcing a continuous, smooth latent space representation



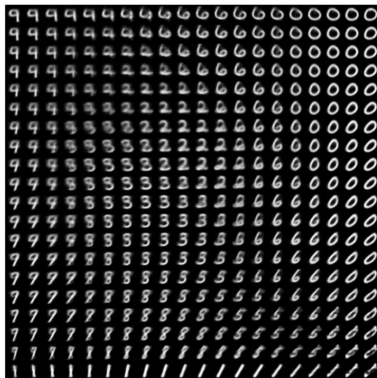
Variational AutoEncoders (VAE)

- ▶ In a VAE there are an encoder ANN and decoder ANN
- ▶ The first ANN maps the input variable to a latent space that corresponds to the **parameters** of a variational **distribution** (e.g. the mean and the covariance)
- ▶ In this way, we can ask the encoder to provide multiple different samples, all coming from the same distribution
- ▶ The decoder has the opposite function, which is to map from the latent space to the input space, so to produce realistic instances
- ▶ The distribution chosen for the variational distribution in the latent space is as close as possible to a multivariate Gaussian



Variational AutoEncoders (VAE)

- ▶ We can "interpolate" between different regions and we will smoothly transition across points that are all mapped into reasonable instances
- ▶ Generally we interpolate following the geodesic on the manifold, which corresponds to arcs on a hyper-sphere



Variational AutoEncoders (VAE)

- ▶ We can "interpolate" between different regions and we will smoothly transition across points that are all mapped into reasonable instances



Outline

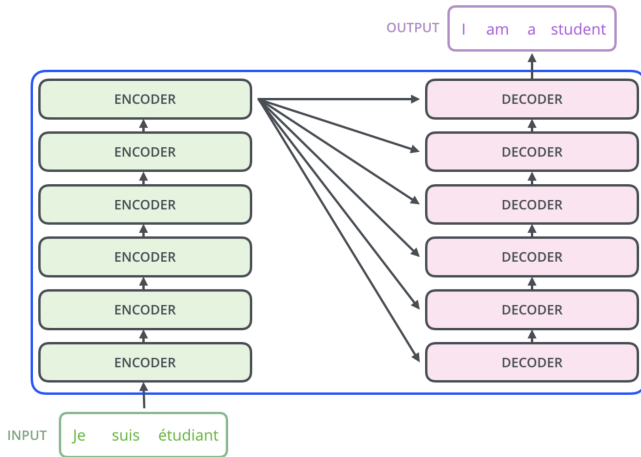
- 1 Generative Systems
- 2 Autoencoders
- 3 Generative Adversarial Networks (GANs)
- 4 Variational AutoEncoders
- 5 Transformers**
- 6 Diffusion models

Transformers (T)

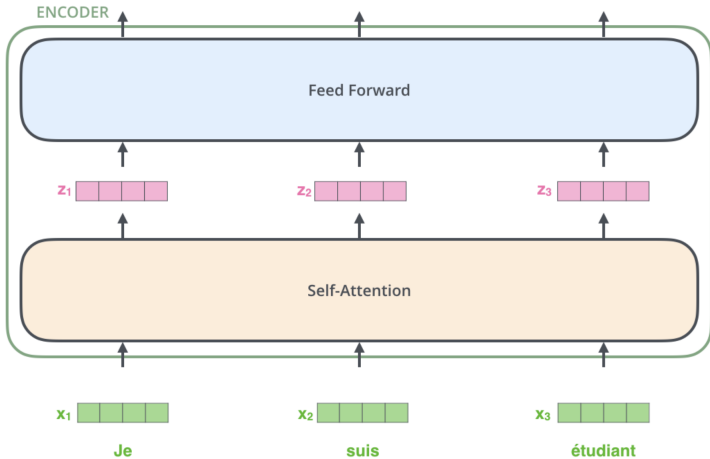
- ▶ The transformer architecture can map sequences to sequences
- ▶ Differently from recurrent architectures, it process the entire input all at once.
- ▶ Transformers were introduced in 2017 by Google and have replaced RNN models such as long short-term memory (LSTM) for NLP problems.
- ▶ Since their training can be parallelised they can be trained on larger datasets.
- ▶ Systems based T are BERT (Bidirectional Encoder Representations from Transformers) and GPT (Generative Pre-trained Transformer)
- ▶ These systems have shown emergent capacities when scaled to very large sizes and datasets (Wikipedia Corpus and Common Crawl)

Transformers (T)

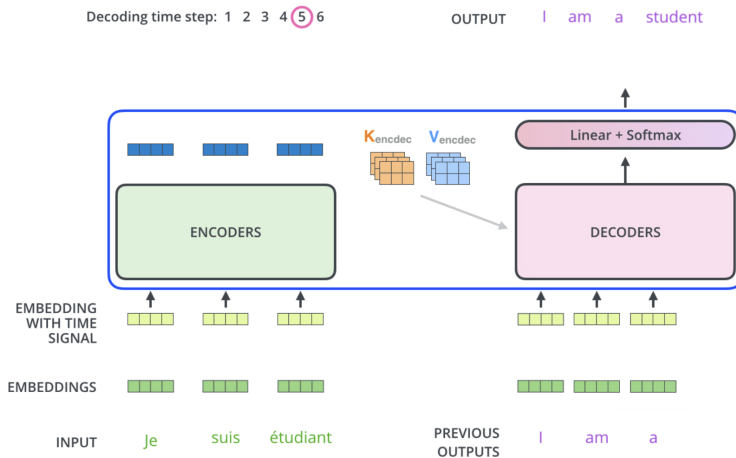
Find detailed explanation at:
<http://jalamar.github.io/illustrated-transformer/>



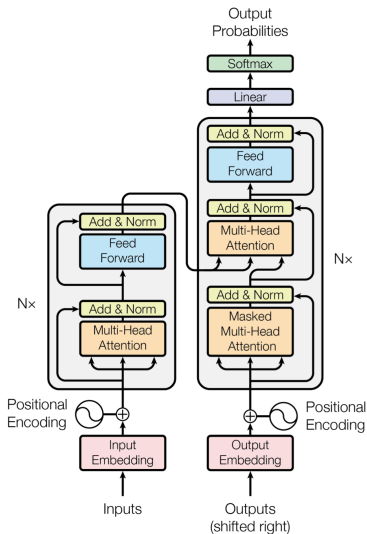
Transformers (T)



Transformers (T)



Transformers (T)

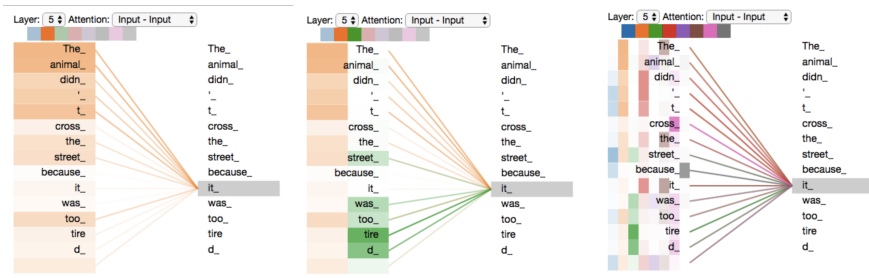


Transformers (T)

- ▶ T uses an encoder-decoder architecture
- ▶ The input text is parsed into tokens, each converted via a word embedding into a vector and augmented by positional information.
- ▶ The encoder (decoder) consists of encoding (decoding) layers that process the input (output) one layer after another
- ▶ Encoding layers generate token encodings and learn which parts of the inputs are relevant to each other
- ▶ Decoding layers take these encodings and generate the output (token) sequence
- ▶ Both encoding and decoding layers makes use of an **attention** mechanism

Attention

- ▶ Transformer layers are scaled dot-product attention units
- ▶ A new embedding is computed for each token as a weighted sum of the other (projected) tokens
- ▶ The contribution of each token is determined by the attention weight
- ▶ The attention weight is computed as the dot product (i.e. how similar) between (projected) encodings of the tokens

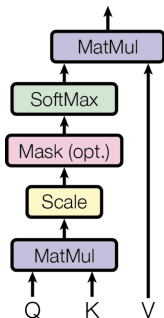


- ▶ For each attention unit, the transformer model learns three weight (projection) matrices; the query weights W_Q , the key weights W_K , and the value weights W_V
- ▶ For each token i , the input word embedding x_i is multiplied with each of the three weight matrices to produce a query vector $q_i = x_i W_Q$, a key vector $k_i = x_i W_K$, and a value vector $v_i = x_i W_V$
- ▶ Attention weights are calculated using the query and key vectors: the attention weight a_{ij} from token i to token j is the dot product between q_i and k_j
- ▶ The fact that W_Q and W_K are different matrices allows attention to be non-symmetric: if token i attends to token j this does not necessarily mean that token j will attend to token i

Attention

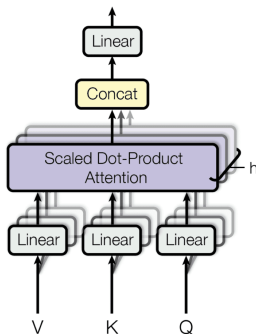
- ▶ The matrices Q , K and V are defined as the matrices where the i th rows are vectors q_i , k_i , and v_i
- ▶ The attention is defined as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$



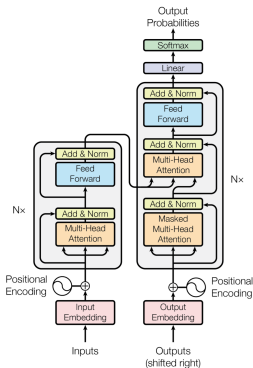
Multi-head attention

- ▶ One set of (W_Q, W_K, W_V) matrices is called an attention head
- ▶ Using multiple attention heads allows to model different types of "relevance"
- ▶ E.g. one attention head attends to the next word, while another attends from verbs to their direct objects
- ▶ The outputs from each attention layer are then all concatenated



Masked attention

- ▶ Note that the decoder for token position t should not have access to token position $t+1$ (it cannot look into the future)
- ▶ This can be enforced before the softmax stage by adding a mask matrix M that is negative infinity at entries where the attention link must be cut, and zero at other places

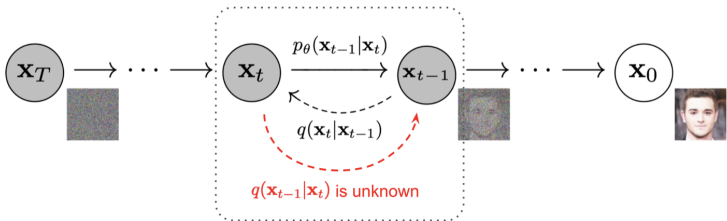


Outline

- 1 Generative Systems
- 2 Autoencoders
- 3 Generative Adversarial Networks (GANs)
- 4 Variational AutoEncoders
- 5 Transformers
- 6 Diffusion models**

Diffusion models (DM)

- ▶ Diffusion models are inspired by non-equilibrium thermodynamics
- ▶ They define a Markov chain of diffusion steps that slowly add random noise to data and then learn to reverse the diffusion process to construct meaningful data samples from pure noise
- ▶ Unlike VAE, diffusion models are learned with a fixed procedure and the latent variable has high dimensionality (same as the original data)



Controlling the output of Diffusion models with text

- ▶ A trained Diffusion Model starts with Gaussian noise and then iteratively generates an image that is similar to the images on which it was trained
- ▶ How can we control what image is actually output?
- ▶ it seems we simply input Gaussian noise into the model, and out comes a random image that looks like it could belong to the training dataset
- ▶ ...we need a method of incorporating a conditioning element into the diffusion process

Diffusion models (DM)

- ▶ We can use a text encoder to produce an encoding of a textual caption
- ▶ This "encoding" is a sequence of vectors
- ▶ To inject this information into the Diffusion Model, we can pool these vectors together and condition our diffusion model on them
- ▶ By conditioning on this vector, the Diffusion Model learns how to adapt its denoising procedure to produce an image that is well-aligned with the caption

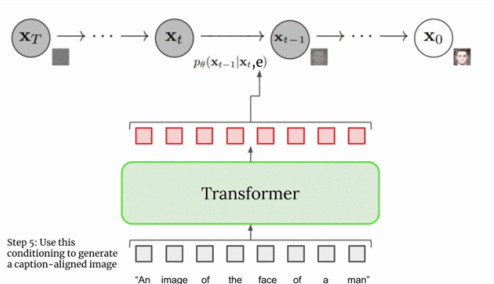
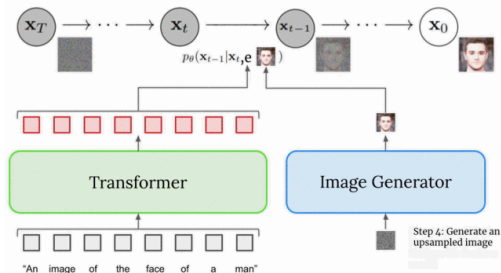


Image Super-Resolution

- ▶ The image generator, or "base" model, outputs a small 64x64 image. To upsample this model to the final 1024x1024 version we use a super-resolution model to intelligently upsample the image
- ▶ For the super-resolution model, we can use another Diffusion Model
- ▶ now, instead of conditioning on just the caption encoding, we also condition on the smaller image which we want to upsample

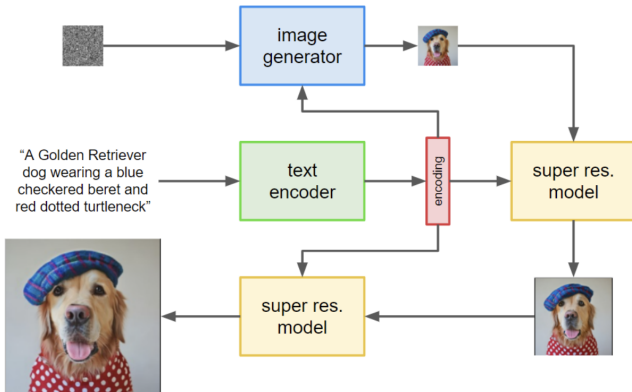


Conditional Diffusion Models

- ▶ To summarize, the caption is input into a pre-trained encoder which outputs a sequence of vectors (the text encoding)
- ▶ These vectors are important because they encode how words in the caption relate to one another and act as conditioning information for all other components of the model
- ▶ The text encodings are then passed into an image generation Diffusion Model, which starts with Gaussian noise and then gradually removes noise to generate a novel image which reflects the semantic information within the caption

Conditional Diffusion Models

- ▶ The output of this model is a 64x64 pixel image. After this, two more Diffusion Models are used to super-resolve this image to the final 1024x1024 size, again conditioned on the text encodings (as well as lower resolution images)



Conditional Diffusion Models Applications

- ▶ It is now possible to condition on parts of the image and selectively change aspects at a very highly abstract level
- ▶ E.g. change location



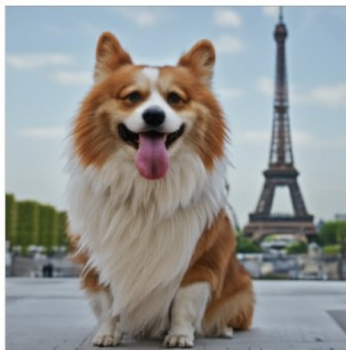
A [V] dog in the
Versailles hall of mirrors



A [V] dog in the
gardens of Versailles



A [V] dog in the
Tokyo bay



A [V] dog in the
Paris square

Conditional Diffusion Models Applications

- ▶ It is now possible to condition on parts of the image and selectively change aspects at a very highly abstract level
- ▶ E.g. change style



Vincent Van Gogh



Michelangelo



Rembrandt



Johannes Vermeer



Pierre-Auguste Renoir



Leonardo da Vinci

Conditional Diffusion Models Applications

- ▶ It is now possible to condition on parts of the image and selectively change aspects at a very highly abstract level
- ▶ E.g. change accessory



Chef Outfit



Witch Outfit



Ironman Outfit



Nurse Outfit



Purple Wizard Outfit



Superman Outfit



Police Outfit



Angel Wings

Conditional Diffusion Models Applications

- ▶ It is now possible to condition on parts of the image and selectively change aspects at a very highly abstract level
- ▶ E.g. change expression

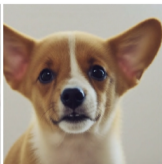
Expression modification (“A [state] [V] dog”)



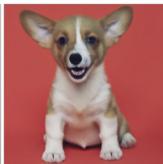
depressed



sleeping



sad



joyous



barking



crying



frowning



screaming

Autoencoders

- ▶ Unsupervised learning of representations.
- ▶ Can be stacked for learning deep hierarchies.
- ▶ Can be used as pre-training regime for a deep network.

Generative Adversarial

- ▶ Train two networks: A Generator and a Discriminator competitively.
- ▶ Can allow to formulate transfer of styles tasks.

Diffusion

- ▶ Starts with Gaussian noise and then iteratively generates an image that is similar to the images on which it was trained
- ▶ Can be conditioned on text prompts